# Discretización y ecuación de diferencias

## Para la programación y simulación de sistemas de control

Laboratorio de Control Automático · EL5409
Prof. Pablo César Rodríguez Vargas

Presentación y simulaciones:
https://github.com/pablocero17/labcontrol-discretizacion-controldigital



# Contenido




**Métodos de discretización** a partir de la función de transferencia continua:

- Respuesta invariante al impulso
- Retenedor de orden cero (ZOH)
- Tustin o transformación bilineal
- Mapeo de polos y ceros (solo SISO)

**Ejemplos que se desarrollan** ($T = 0.5$ s):

**Ejemplo 1**: $G(s) = \dfrac{1}{s+1}$

**Ejemplo 2**: $G(s) = \dfrac{2}{s^2 + 0.4s + 2}$

Cada uno: discretización ▶ ecuación de diferencias ▶ MATLAB ▶ Arduino.

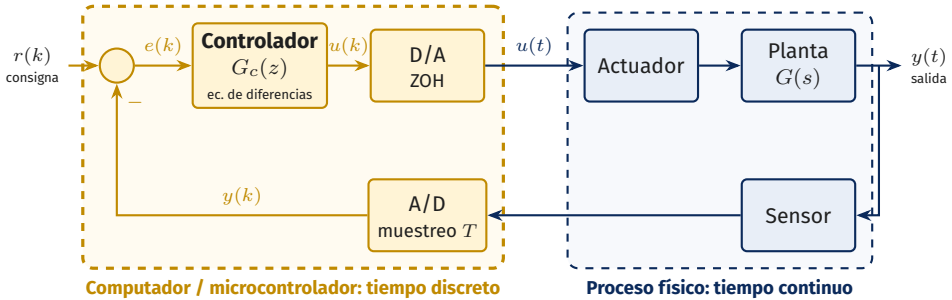
Tema 1

# Fundamentos del control digital

Esquema del lazo, periodo de muestreo y señales discretas



# Esquema de un control digital




**A/D**: el sensor se lee solo en $t = kT$. El controlador ve la secuencia $y(k)$.

**Controlador**: una **ecuación de diferencias** que calcula $u(k)$ con muestras actuales y pasadas.

**D/A con ZOH**: $u(k)$ se mantiene constante durante todo el periodo (como el PWM).

# Escogencia del periodo de muestreo

**Regla general**

$$T_s \;<\; \frac{1}{10}\,T_{\text{dom}}$$

$T_{\text{dom}}$: constante de tiempo dominante del sistema que se espera **en lazo cerrado**.

$T$ **grande**: se pierde información entre muestras y el lazo se acerca a la inestabilidad.
$T$ **muy pequeño**: coeficientes muy cercanos a 1, más ruido en la derivada y más carga de cálculo.

| **Criterio** | Rango recomendado |
|---|---|
| **Tiempo de subida** $t_r$ | $t_r/20 < T < t_r/10$ |
| **Tiempo de establ.** $t_s$ | $t_s/75 < T < t_s/25$ |
| **Ancho de banda** $BW$ **[Hz]** | $20\,BW < f_T < 40\,BW$ |
| **Frec. amortiguada** $\omega_d$ | $30\,\omega_d < \omega_T < 60\,\omega_d$ |
| **Lazo abierto, cruce** $\omega_{cg}$ | $40\,\omega_{cg} < \omega_T < 80\,\omega_{cg}$ |

$f_T = 1/T$, $\omega_T = 2\pi/T$.

**En los ejemplos**: Ej. 1 ($\tau = 1$ s) pide $T < 0.1$ s; Ej. 2 ($\omega_d = 1.4$ rad/s) pide $T < 0.15$ s. Se usa $T = 0.5$ s **a propósito**, para que se vean los escalones.

# Representación de señales discretas

**Transformada** $Z$ de una secuencia:

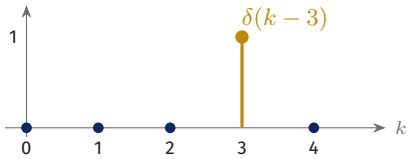
$$X(z) = \sum_{k=0}^{\infty} x(k)\, z^{-k}$$

Cada potencia $z^{-n}$ marca **en qué muestra** está el valor:

$$X(z) = 0 + 0z^{-1} + 0z^{-2} + 1z^{-3} + 0z^{-4}$$

$$X(z) = z^{-3} \longleftrightarrow x(k) = \delta(k-3)$$

$z^{-1}$ **es un retardo de una muestra**: multiplicar por $z^{-n}$ equivale a tomar el valor de $n$ periodos atrás.




| Dominio $z$ | | Muestras |
|---|---|---|
| $4z^{-3}$ | $\leftrightarrow$ | $4x(k-3)$ |
| $4z^{-3} + 7z^{-2} - 2z^{-1}$ | $\leftrightarrow$ | $4x(k-3) + 7x(k-2)$ |
| $z^{-n}X(z)$ | $\leftrightarrow$ | $x(k-n)$ |

**Esta regla es la base de la ecuación de diferencias.**

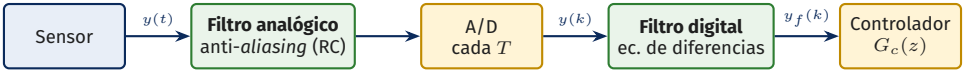
Tema 2

# Filtrado de la medición

Reducir el ruido del sensor antes y después de muestrear



# ¿Dónde se filtra en un lazo digital?




**Antes del A/D (hardware)**: un pasa bajos RC con corte menor que $f_T/2$ (frecuencia de Nyquist) evita el *aliasing*: el ruido rápido no se "disfraza" de señal lenta.

**Después del A/D (software)**: el filtro también es una ecuación de diferencias. En tiempo real solo se pueden usar filtros **causales** (con muestras actuales y pasadas).

Todo filtro causal **agrega retraso** a la medición: el controlador reacciona más tarde. Se debe escoger el filtro más suave que resuelva el problema.

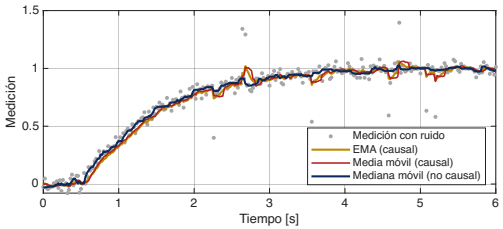
# Tipos de filtros recomendados

| Filtro | Ecuación / idea | MATLAB | ¿Tiempo real? |
|---|---|---|---|
| **Exponencial (EMA)** | $y_f(k) = \alpha\, y(k) + (1-\alpha)\, y_f(k-1)$. Pasa bajos RC discretizado. | `filter(a,[1 a-1],y)` | ✔ |
| **Media móvil** | Promedio de las últimas $n$ muestras (FIR). Ruido aleatorio. | `filter(ones(1,n)/n,1,y)` | ✔ |
| **Pasa bajos Butterworth** | 2.º orden o más; corte más definido. Se diseña en $s$ y se discretiza con Tustin. | `c2d(F,T,'tustin')` | ✔ |
| **Mediana móvil** | Mediana de $n$ muestras: **elimina picos** aislados. | `movmedian(y,n)` | ∼ |
| **Savitzky-Golay** | Ajuste polinomial local: suaviza y **conserva la forma**. | `smoothdata(y,'sgolay')` | ✘ |
| **Filtro de fase cero** | Filtra hacia adelante y hacia atrás: sin retraso. | `filtfilt(b,a,y)` | ✘ |
| **Filtro de Kalman** | Estima el estado con el modelo y la medición. Avanzado. | `kalman(sys,Q,R)` | ✔ |

✔ causal, programable en el micro · ∼ causal con retraso de $n/2$ muestras · ✘ solo para datos registrados (análisis, identificación). `filtfilt` requiere Signal Processing Toolbox.

**Recomendación práctica**: EMA para el controlador ($\alpha$ entre 0.1 y 0.5), mediana si hay picos, y filtros de fase cero solo para procesar datos ya guardados.

# Filtros aplicados a la medición del Ejemplo 1




matlab/filtros_senal.m

```
%% 1) Filtros causales: se pueden programar en el microcontrolador
alfa = 0.2;
y_ema = filter(alfa, [1, alfa-1], y);          % EMA: yf(k) = a y(k) + (1-a) yf(k-1)
n = 7;
y_mm = filter(ones(1, n)/n, 1, y);             % media móvil de n muestras (FIR)

%% 2) Filtros no causales (fase cero): solo para datos ya registrados
y_med = movmedian(y, n);                       % mediana móvil: elimina picos
y_sg  = smoothdata(y, 'sgolay', 15);           % Savitzky-Golay: conserva la forma
```

**El EMA es un RC discretizado**: con $\alpha = 0.2$ y $T_s = 20$ ms equivale a $\tau_f = 0.09$ s ($f_c = 1.8$ Hz). `c2d(1/(0.09s+1), 0.02, 'zoh')` da exactamente $0.2/(z - 0.8)$.

Tema 3

# Métodos de discretización

De la función de transferencia continua $G(s)$ a $G(z)$



# Métodos de discretización en MATLAB: `Gz = c2d(G, T, 'método')`

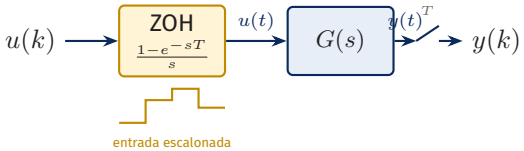
| Método | **Nombre** | Idea | Uso recomendado |
|---|---|---|---|
| `'zoh'` | **Retenedor de orden cero** | $G(z) = (1 - z^{-1})\,\mathcal{Z}\{G(s)/s\}$. Entrada constante entre muestras. Exacto en los instantes $kT$. | **Plantas** (D/A, PWM) |
| `'impulse'` | **Invariante al impulso** | $g_d(k) = T\,g(kT)$: iguala la respuesta al impulso. | Filtros, señales |
| `'tustin'` | **Bilineal (Tustin)** | $s \leftarrow \dfrac{2}{T}\dfrac{z-1}{z+1}$: integra con trapecios. Conserva la estabilidad. | **Controladores** y filtros |
| `'matched'` | **Mapeo de polos y ceros** | $z = e^{sT}$ para polos y ceros; ajusta la ganancia DC. Solo SISO. | Diseño por polos |
| `'foh'` | **Retenedor de 1.er orden** | La entrada varía en rampa entre muestras. | Actuadores suaves |
| `'least-squares'` | **Mínimos cuadrados** | Ajusta la respuesta en frecuencia discreta a la continua. | Alta fidelidad en frecuencia |

**Tustin con pre-distorsión**: exacto en una frecuencia de interés $\omega_c$.

```
opc = c2dOptions('Method','tustin','PrewarpFrequency',wc);
Gz = c2d(G, T, opc);
```

**Regla rápida**: `'zoh'` para la planta (así actúa el D/A) y `'tustin'` para el controlador. `d2c(Gz)` hace el camino inverso.

# Discretización por retenedor de orden cero (ZOH)




$$G(z) = \mathcal{Z}\left\{\frac{1-e^{-sT}}{s}\,G(s)\right\}$$
$$= (1-z^{-1})\,\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

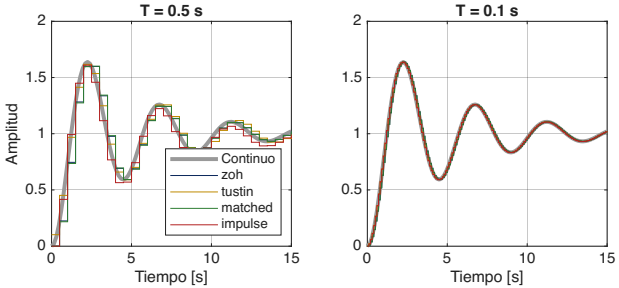
## Procedimiento

1. Combinar la función del retenedor con la de la planta: $G(s)/s$.
2. Descomponer en **fracciones parciales**.
3. Transformar cada fracción a $Z$ con la tabla.
4. Multiplicar por $(1-z^{-1}) = \dfrac{z-1}{z}$.

**Tabla mínima**:

$$\mathcal{Z}\left\{\frac{1}{s}\right\} = \frac{z}{z-1}$$
$$\mathcal{Z}\left\{\frac{1}{s+a}\right\} = \frac{z}{z-e^{-aT}}$$

# Comparación de métodos con la planta del Ejemplo 2




| método | num | den |
|---|---|---|
| zoh | [0 0.2247 0.2099] | [1 -1.384 0.8187] |
| tustin | [0.102 0.204 0.102] | [1 -1.429 0.8367] |
| matched | [0 0.2173 0.2173] | [1 -1.384 0.8187] |
| impulse | [0 0.4164 0] | [1 -1.384 0.8187] |

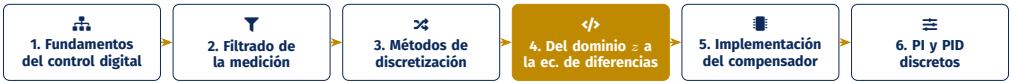
- `zoh`, `matched` e `impulse` dan los **mismos polos** ($e^{p_i T}$); cambian los ceros.
- `tustin` desplaza un poco los polos.
- Con $T = 0.1$ s todos coinciden con la respuesta continua.

Escoger bien $T$ importa más que el método.

Tema 4

# Del dominio $z$ a la ecuación de diferencias

El algoritmo que se programa, con los ejemplos 1 y 2



# Procedimiento: de $G(z)$ a la ecuación de diferencias

**1. Potencias negativas**

Dividir num. y den. entre la mayor potencia de $z$

$$\frac{b_1 z^{-1} + b_2 z^{-2}}{1 - a_1 z^{-1} + a_2 z^{-2}}$$

**2. Multiplicar en cruz**

$$X(z)\left[1 - a_1 z^{-1} + a_2 z^{-2}\right] = U(z)\left[b_1 z^{-1} + b_2 z^{-2}\right]$$

**3. Transformada inversa**

Cada $z^{-n} X(z)$ se cambia por $x(k-n)$

$$x(k) - a_1 x(k-1) + a_2 x(k-2) = b_1 u(k-1) + b_2 u(k-2)$$

**4. Despejar $x(k)$**

$$x(k) = b_1 u(k-1) + b_2 u(k-2) + a_1 x(k-1) - a_2 x(k-2)$$

el algoritmo que se programa

**En MATLAB**: `[num, den] = tfdata(Gz, 'v')` entrega los coeficientes. El coeficiente de $x(k-n)$ es $-$**den**$(n+1)$: cambia de signo al pasar al otro lado.

**Interpretación**: la salida actual se calcula con entradas pasadas y salidas pasadas. Solo hay que **guardar** esos valores entre un periodo y el siguiente.

# Ejemplo 1: discretización de un sistema de primer orden

Planta $G(s) = \dfrac{1}{s+1}$, retenedor ZOH, $T = 0.5$ s:

$$\begin{aligned}\frac{X(z)}{U(z)} &= (1-z^{-1})\,\mathcal{Z}\left\{\frac{1}{s(s+1)}\right\} \\ &= (1-z^{-1})\,\mathcal{Z}\left\{\frac{1}{s} - \frac{1}{s+1}\right\} \\ &= \frac{z-1}{z}\left[\frac{z}{z-1} - \frac{z}{z-e^{-T}}\right] = \frac{1-e^{-T}}{z-e^{-T}}\end{aligned}$$

Con $e^{-0.5} = 0.6065$:

$$\frac{X(z)}{U(z)} = \frac{0.3935}{z-0.6065} = \frac{0.3935\,z^{-1}}{1-0.6065\,z^{-1}}$$

## A la ecuación de diferencias

$$(1-0.6065z^{-1})\,X(z) = 0.3935z^{-1}\,U(z)$$

$$X(z) = 0.3935z^{-1}U(z) + 0.6065z^{-1}X(z)$$

$$x(k) = 0.3935\,u(k-1) + 0.6065\,x(k-1)$$

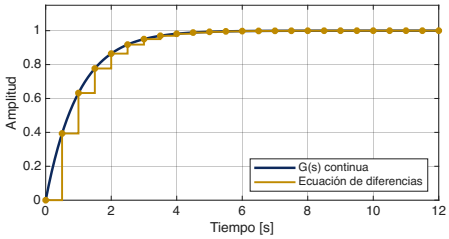
| $k$ | cálculo (escalón $u=1$) | $x(k)$ |
|---|---|---|
| 0 | condiciones iniciales nulas | 0 |
| 1 | $0.3935(1) + 0.6065(0)$ | 0.3935 |
| 2 | $0.3935(1) + 0.6065(0.3935)$ | 0.6322 |
| 3 | $0.3935(1) + 0.6065(0.6322)$ | 0.7769 |

Valor final: $\dfrac{0.3935}{1-0.6065} = 1$ ✓

# Ejemplo 1: simulación en MATLAB

matlab/ej1_primer_orden.m

```
3  s = tf('s');
4  G = 1/(s + 1);                      % planta en tiempo continuo
5  T = 0.5;                            % periodo de muestreo [s]
6
7   %% 1) Discretizar con retenedor de orden cero (ZOH)
8  Gz = c2d(G, T, 'zoh')               % resultado: 0.3935 / (z - 0.6065)
9  [num, den] = tfdata(Gz, 'v');       % num = [0 0.3935]   den = [1
       -0.6065]
10
11  %% 2) Ecuación de diferencias:  x(k) = 0.3935 u(k-1) + 0.6065 x(k-1)
12 b1 = num(2);                        % coeficiente de u(k-1)
13 a1 = -den(2);                       % coeficiente de x(k-1)  (cambia de
       signo)
14 fprintf('x(k) = %.4f u(k-1) + %.4f x(k-1)\n', b1, a1);
15
16 tsim = 12;  k = 0:T:tsim;  N = numel(k);
17 u = ones(1, N);                     % escalón unitario
18 x = zeros(1, N);
19 for n = 1:N
20     x1 = 0;  u1 = 0;                % muestras anteriores (cero antes de
           empezar)
21     if n > 1, x1 = x(n-1);  u1 = u(n-1);  end
22     x(n) = b1*u1 + a1*x1;           % <- ecuación de diferencias
23 end
```




**Línea 8**: `c2d` entrega $0.3935/(z - 0.6065)$. **Líneas 12–13**: se leen $b_1$ y $a_1$ con `tfdata`. **Línea 22**: la ecuación de diferencias. Los puntos caen **sobre** la curva continua en cada $kT$.

# Ejemplo 2: discretización de un sistema de segundo orden

Planta $G(s) = \dfrac{2}{s^2 + 0.4s + 2}$, retenedor ZOH, $T = 0.5$ s:

$$\frac{X(z)}{U(z)} = (1 - z^{-1})\,\mathcal{Z}\left\{\frac{1}{s} \cdot \frac{2}{s^2 + 0.4s + 2}\right\}$$

Polos complejos $s = -0.2 \pm j1.40$ ($\omega_n = 1.414$ rad/s, $\zeta = 0.14$). Las fracciones parciales se resuelven con la tabla, o directamente con `c2d`:

$$\begin{aligned}\frac{X(z)}{U(z)} &= \frac{0.2247z + 0.2099}{z^2 - 1.384z + 0.8187} \\ &= \frac{0.2247z^{-1} + 0.2099z^{-2}}{1 - 1.384z^{-1} + 0.8187z^{-2}}\end{aligned}$$

Polos de $G(z)$: $z = e^{(-0.2 \pm j1.40)(0.5)}$,
$|z| = e^{-0.1} = 0.905 < 1$: estable.

## A la ecuación de diferencias

$$\begin{aligned}&(1 - 1.384z^{-1} + 0.8187z^{-2})\,X(z) \\ &= (0.2247z^{-1} + 0.2099z^{-2})\,U(z)\end{aligned}$$

$$\begin{aligned}x(k) = {} & 0.2247\,u(k-1) + 0.2099\,u(k-2) \\ & + 1.384\,x(k-1) - 0.8187\,x(k-2)\end{aligned}$$

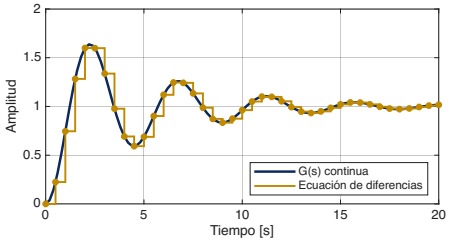
| $k$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $x(k)$ | 0 | 0.2247 | 0.7456 | 1.2826 | 1.5995 |

Valor final: $\dfrac{0.2247 + 0.2099}{1 - 1.384 + 0.8187} = 1$ ✓

# Ejemplo 2: simulación en MATLAB

matlab/ej2_segundo_orden.m

```
7  %% 1) Discretizar con retenedor de orden cero (ZOH)
8  Gz = c2d(G, T, 'zoh')              % (0.2247 z + 0.2099) / (z^2 - 1.384
         z + 0.8187)
9  [num, den] = tfdata(Gz, 'v');      % num = [0 b1 b2]   den = [1 -a1 a2]
10
11 %% 2) Ecuación de diferencias
12 %   x(k) = b1 u(k-1) + b2 u(k-2) + a1 x(k-1) - a2 x(k-2)
13 b1 = num(2);  b2 = num(3);
14 a1 = -den(2);  a2 = den(3);
15 fprintf('x(k) = %.4f u(k-1) + %.4f u(k-2) + %.4f x(k-1) - %.4f x(k-2)\
       n', b1, b2, a1, a2);
16
17 tsim = 20;  k = 0:T:tsim;  N = numel(k);
18 u = ones(1, N);                    % escalón unitario
19 x = zeros(1, N);
20 for n = 1:N
21     x1 = 0;  x2 = 0;  u1 = 0;  u2 = 0;          % muestras anteriores
22     if n > 1, x1 = x(n-1);  u1 = u(n-1);  end
23     if n > 2, x2 = x(n-2);  u2 = u(n-2);  end
24     x(n) = b1*u1 + b2*u2 + a1*x1 - a2*x2;       % <- ecuación de
           diferencias
25 end
```




Las memorias `x1`, `x2`, `u1`, `u2` son $x(k-1)$, $x(k-2)$, $u(k-1)$, $u(k-2)$; valen cero antes de empezar (líneas 21–23). La línea 24 es la ecuación de diferencias. El `for` coincide con `lsim(Gz,u,k)` (diferencia $< 10^{-15}$).

# Simulación en Arduino: la misma ecuación en el microcontrolador

arduino/ej2_segundo_orden/ej2_segundo_orden.ino

```
9  void loop() {
10   if (millis() - tAnterior < T_MS) return;   // esperar a que pase T
11   tAnterior += T_MS;
12
13   float u = 1.0;                                          // escalón
         unitario
14   float x = b1 * u_1 + b2 * u_2 + a1 * x_1 - a2 * x_2;   // <- ecuación
         de diferencias
15   x_2 = x_1;  x_1 = x;                                    // correr las
         memorias
16   u_2 = u_1;  u_1 = u;
17
18   Serial.print(k); Serial.print(',');
19   Serial.print(u, 1); Serial.print(',');
20   Serial.println(x, 4);
21   k++;
```

**Correr las memorias** (líneas 25–26): primero $x(k-2) \leftarrow x(k-1)$ y después $x(k-1) \leftarrow x(k)$. Si se invierte el orden, se pierde $x(k-1)$.

**Monitor serie** (`k,u,x`):

```
0,1.0,0.0000
1,1.0,0.2247
2,1.0,0.7456
3,1.0,1.2826    = MATLAB
```

**Tinkercad (simulación en línea)**:

- Sistema de primer orden
- Sistema de segundo orden

`ej1_primer_orden.ino` es igual, con una sola memoria de cada señal. Usar el *Serial Plotter* para ver la curva.

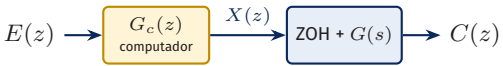
Tema 5

# Implementación digital del compensador

De $G_c(z)$ a un algoritmo realizable



# Implementación digital de un compensador $G_c(z)$




$$G_c(z) = \frac{X(z)}{E(z)} = \frac{a_3 z^3 + a_2 z^2 + a_1 z + a_0}{b_2 z^2 + b_1 z + b_0}$$

**1. Multiplicar en cruz:**

$$(b_2 z^2 + b_1 z + b_0)\, X(z) = (a_3 z^3 + a_2 z^2 + a_1 z + a_0)\, E(z)$$

**2. Despejar el término de mayor orden de $X(z)$ y dividir entre $b_2 z^2$:**

$$X(z) = \left(\tfrac{a_3}{b_2} z + \tfrac{a_2}{b_2} + \tfrac{a_1}{b_2} z^{-1} + \tfrac{a_0}{b_2} z^{-2}\right) E(z) - \left(\tfrac{b_1}{b_2} z^{-1} + \tfrac{b_0}{b_2} z^{-2}\right) X(z)$$

**3. Transformada inversa** ($z^{-n} \rightarrow$ retardo de $n$ muestras):

$$x(k) = \tfrac{a_3}{b_2} e(k+1) + \tfrac{a_2}{b_2} e(k) + \tfrac{a_1}{b_2} e(k-1) + \tfrac{a_0}{b_2} e(k-2) - \tfrac{b_1}{b_2} x(k-1) - \tfrac{b_0}{b_2} x(k-2)$$

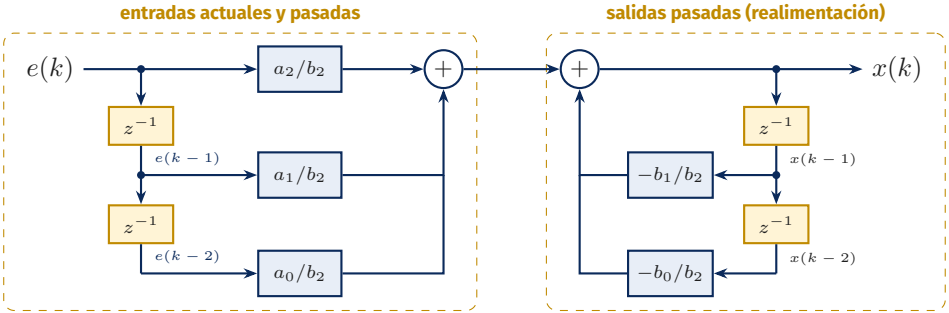
**Realizabilidad física**: $x(k)$ no puede depender del valor **futuro** $e(k+1)$. Por eso $a_3 = 0$.

Regla: **grado del numerador $\leq$ grado del denominador**.

Algoritmo realizable: entradas actuales y pasadas, salidas pasadas.

$$x(k) = \tfrac{a_2}{b_2} e(k) + \tfrac{a_1}{b_2} e(k-1) + \tfrac{a_0}{b_2} e(k-2) - \tfrac{b_1}{b_2} x(k-1) - \tfrac{b_0}{b_2} x(k-2)$$

# Diagrama de bloques de la implementación (forma directa)




Cada bloque $z^{-1}$ es un **retardo de $T$ segundos**: en el código es una variable que guarda el valor del periodo anterior.

El número de memorias es el **orden** del compensador: 2 de la entrada y 2 de la salida para un sistema de segundo orden.

Tema 6

# PI y PID discretos

Del regulador continuo a la ecuación de diferencias



# El PID ideal

En el dominio del tiempo:

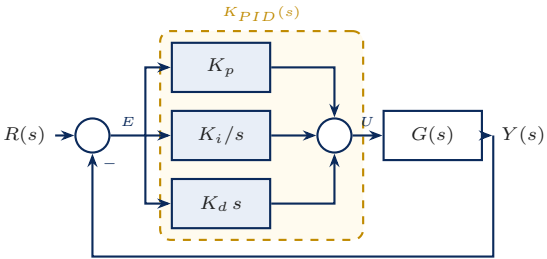
$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d\, \frac{de(t)}{dt}$$

En el dominio de $s$:

$$K_{PID}(s) = K_p + \frac{K_i}{s} + K_d s$$

$$K_{PID}(s) \;=\; K_p \left(1 + \frac{1}{T_i\, s} + T_d\, s\right)$$

$K_i = K_p/T_i, \quad K_d = K_p T_d.$



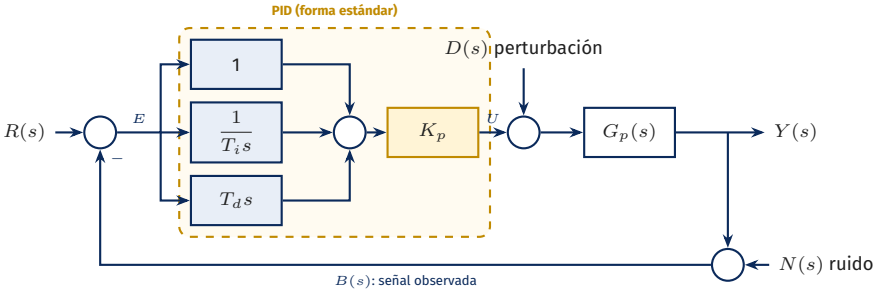

**P**: reacciona al error actual. **I**: elimina el error en estado estable. **D**: anticipa la tendencia y amortigua.

# Casos del regulador PID

| Regulador | Función de transferencia teórica | Forma práctica | Efecto principal |
|---|---|---|---|
| **P** | $K_p$ | $K_p$ | Rápido; deja error en estado estable |
| **I** | $\frac{K_i}{s}$ | $\frac{K_i}{s}$ | Error nulo; lento |
| **PI** | $K_p + \frac{K_i}{s}$ | $K_p \, \frac{s + K_i/K_p}{s}$ | Error nulo y buena rapidez |
| **PD** | $K_p + K_d s$ | $K_p + \frac{K_d \, s}{T_f s + 1}$ | Amortigua; la derivada se filtra |
| **PID** | $K_p + \frac{K_i}{s} + K_d s$ | $K_p + \frac{K_i}{s} + \frac{K_d \, s}{T_f s + 1}$ | Completo |

La **derivada pura** no es realizable y amplifica el ruido: en la práctica se agrega un **filtro de primer orden** ($T_f \approx T_d/10$). En MATLAB: `pid(Kp, Ki, Kd, Tf)`.

# Esquemas de control PID




**Perturbación** $D$: entra a la planta; la acción integral la compensa en estado estable.

**Ruido** $N$: entra por el sensor; la derivada lo amplifica (filtrar la medición, tema 2).

**Patada derivativa**: un escalón en $R$ deriva a un pulso estrecho en $u$. Solución: derivar la medición $-T_d\, s\, Y$ (**PI-D**).

# PI discreto

**Tiempo continuo**

$$U(s) = K_p \left(1 + \frac{1}{T_i\, s}\right) E(s)$$

**Tiempo discreto**: integral por trapecios (Tustin), $\frac{1}{s} \rightarrow \frac{T}{2}\frac{1+z^{-1}}{1-z^{-1}}$

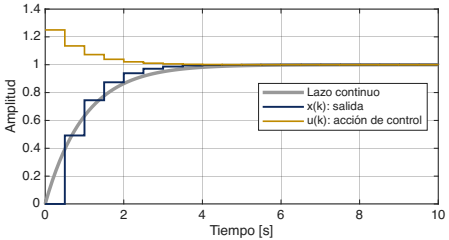
$$U(z) = K_p \left(1 + \frac{T}{2T_i}\,\frac{1+z^{-1}}{1-z^{-1}}\right) E(z)$$

**Ecuación de diferencias**: multiplicar por $(1 - z^{-1})$ y pasar al tiempo

$$u(k) = u(k-1) + K_p \left[\left(1 + \tfrac{T}{2T_i}\right)e(k) - \left(1 - \tfrac{T}{2T_i}\right)e(k-1)\right]$$

$$u(k) = u(k-1) + A\, e(k) + B\, e(k-1)$$

$$A = K_p\left(1 + \frac{T}{2T_i}\right) \qquad B = -K_p\left(1 - \frac{T}{2T_i}\right)$$




**Ejemplo**: planta del Ejemplo 1, $K_p = 1$, $T_i = 1$ s, $T = 0.5$ s: $A = 1.25$, $B = -0.75$.
$T_i = \tau$ cancela el polo de la planta: el lazo cerrado se comporta como $1/(s+1)$.

# PID discreto

**Tiempo continuo**

$$U(s) = K_p \left(1 + \frac{1}{T_i s} + T_d\, s\right) E(s)$$

**Tiempo discreto**: integral por trapecios y derivada hacia atrás, $s \to \frac{1-z^{-1}}{T}$

$$U(z) = K_p \left(1 + \frac{T}{2T_i}\frac{1+z^{-1}}{1-z^{-1}} + \frac{T_d}{T}(1-z^{-1})\right) E(z)$$

**Ecuación de diferencias**

$$u(k) = u(k-1) + A\, e(k) + B\, e(k-1) + C\, e(k-2)$$

$$A = K_p\left(1 + \frac{T}{2T_i} + \frac{T_d}{T}\right) \qquad C = K_p \frac{T_d}{T}$$

$$B = -K_p\left(1 - \frac{T}{2T_i} + \frac{2T_d}{T}\right)$$

**Forma incremental**: se calcula el *cambio* de $u$. Si $u$ se limita (saturación del actuador), la integral no se acumula de más (anti-*windup* natural).

**Ejemplo**: $K_p = 1$, $T_i = 1$ s, $T_d = 0.25$ s, $T = 0.5$ s

$A = 1.75 \quad B = -1.75 \quad C = 0.5$

$C = K_p T_d / T$ crece si $T$ disminuye: la derivada es la parte más sensible al ruido. Filtrar la medición (tema 2) o usar la derivada filtrada.

# PI y PID discretos en MATLAB y en Arduino

matlab/pi_pid_discreto.m

```
36 Td = 0.25;
37 A = Kp*(1 + T/(2*Ti) + Td/T);
38 B = -Kp*(1 - T/(2*Ti) + 2*Td/T);
39 C = Kp*Td/T;
40 fprintf('PID: u(k) = u(k-1) + %.4f e(k)  %+.4f e(k-1)  %+.4f e(k
       -2)\n', A, B, C);
41 Cpid = pid(Kp, Kp/Ti, Kp*Td, 'Ts', T, 'IFormula', 'Trapezoidal',
        'DFormula', 'BackwardEuler');
42 [q, ~] = tfdata(tf(Cpid), 'v');
43 fprintf('     tfdata(pid):  [%.4f  %.4f  %.4f]\n', q(1), q(2), q
       (3));
```

```
PID: u(k) = u(k-1) + 1.7500 e(k) -1.7500 e(k-1) +0.5000 e(k-2)
     tfdata(pid):  [1.7500 -1.7500 0.5000]
```

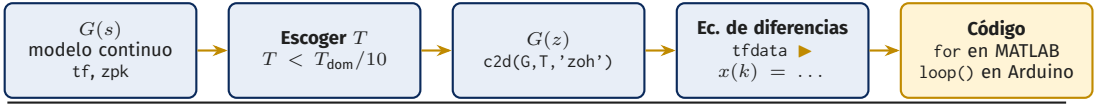
El objeto `pid` con `'Ts'` y las fórmulas `Trapezoidal` / `BackwardEuler` da los mismos $A, B, C$ calculados a mano.

arduino/pi_discreto/pi_discreto.ino

```
22 void loop() {
23   if (millis() - tAnterior < T_MS) return;   // esperar a que
         pase T
24   tAnterior += T_MS;
25
26   x = 0.3935 * u_1 + 0.6065 * x;             // 1) medir (
       planta virtual)
27   float e = r - x;                           // 2) error
28   float u = u_1 + A * e + B * e_1;           // 3) PI: ecuaci
       ón de diferencias
29   u = constrain(u, U_MIN, U_MAX);            // 4) límites
       del actuador
30   e_1 = e;  u_1 = u;                         // 5) memorias
```

**Cada periodo**: medir ▶ error ▶ ecuación del PI ▶ limitar ▶ guardar memorias. Con una planta real, la línea 26 se cambia por `analogRead` y se agrega `analogWrite(u)`.

# Síntesis: del modelo continuo al código




| Caso | Función discreta | Ecuación de diferencias |
|---|---|---|
| **Ejemplo 1** | $\dfrac{0.3935}{z-0.6065}$ | $x(k) = 0.3935\,u(k-1) + 0.6065\,x(k-1)$ |
| **Ejemplo 2** | $\dfrac{0.2247z+0.2099}{z^2-1.384z+0.8187}$ | $x(k) = 0.2247u(k-1)+0.2099u(k-2)+1.384x(k-1)-0.8187x(k-2)$ |
| **PI** | $K_p\left(1+\frac{T}{2T_i}\frac{1+z^{-1}}{1-z^{-1}}\right)$ | $u(k) = u(k-1) + A\,e(k) + B\,e(k-1)$ |
| **PID** | PI $+\ K_p\frac{T_d}{T}(1-z^{-1})$ | $u(k) = u(k-1) + A\,e(k) + B\,e(k-1) + C\,e(k-2)$ |

# Videos y bibliografía de referencia

## Videos

- Sistemas de Control Digital: Controlador PID Digital
  youtu.be/q90q4TYOpwA
- Diseño completo de un controlador PID Digital / PID Arduino
  youtu.be/F9vyPtHhjQ8
- PI y PID discreto en Arduino
  youtu.be/EEuB2GI9s0o
- Implementación de PID discreto en planta física
  youtu.be/iD-AoALbGH0

## Bibliografía


- K. Ogata, *Sistemas de control en tiempo discreto*, 2.ª ed. Prentice Hall, 1996.
- C. L. Phillips y H. T. Nagle, *Digital Control System Analysis and Design*, 4.ª ed. Pearson, 2015.
- K. J. Åström y T. Hägglund, *Advanced PID Control*. ISA, 2006.
- MathWorks, documentación de `c2d` (*Continuous-Discrete Conversion Methods*) y `pid`.

# Discretización y ecuación de diferencias

## Preguntas y discusión




**Descargue la presentación y las simulaciones**

`https://github.com/pablocero17/labcontrol-discretizacion-controldigital`

Botón **Code** ▶ **Download ZIP** · `matlab/` (ejemplos 1 y 2, métodos de `c2d`, filtros, PI/PID) · `arduino/`